Group 09: FPGA Limit Order Book

15-Minute Final Presentation Defense Script & Study Guide

RV32IM RISC-V Softcore + Pure-Hardware Limit Order Book Accelerator on Artix-7 FPGA

Presentation Quick Reference

- **Target Board:** Digilent Nexys A7-100T (Xilinx Artix-7 XC7A100T-1CSG324C).
- **Total Slides:** 12 Slides (Full deck timing budget: 15 minutes exact).
- **Your Core Role (Ronit): Hardware System Bus Architect** (‘sys_bus_interconnect.v‘, 4-state wait-state FSM, MMIO decoder at ‘0x4000_0000‘, Shadow BBO registers, VGA core).
- **Accelerator Highlights:** 6-cycle ADD (60 ns), 8-10 cycle EXEC/CANCEL, O(1) 3-tier BBO priority tree (2-3 cycles vs 8,192 cycles in software).
- **Processor Highlights:** 3-stage core, 3-cycle DSP48E1 MUL, 33-cycle Radix-2 serial DIV, EX-to-EX forwarding, skid buffer.

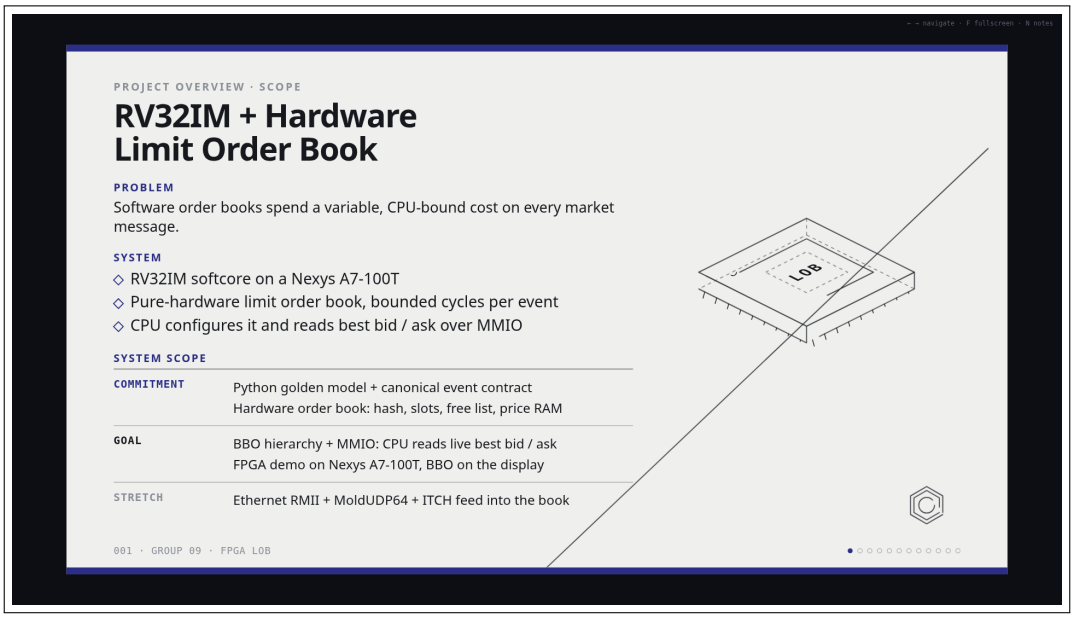
Member Ownership Matrix

Member	Role	Primary Deliverable
Ronit	**Hardware Bus Architect**	**System bus, MMIO arbiter, shadow registers, VGA**
Akshit	RV32M Datapath	DSP48E1 multiplier, Radix-2 divider, hazard unit
Aditya K	LOB Datapath	Canonical event decoder, slot table, price RAM
Aditya M	Priority Encoders	BRAM queues, O(1) hierarchical BBO priority tree
Aditeya G	Bare-Metal Drivers	5 synthetic C workloads, UART, 7-segment display

Slide 1: Project Overview & Scope

RV32IM + Pure-Hardware Limit Order Book Accelerator

Target Time: 1:00



Word-for-Word Spoken Presentation Script

Good morning, professors. Today, Group 09 presents our project: an RV32IM RISC-V softcore processor integrated with a pure-hardware Limit Order Book accelerator on the Artix-7 FPGA.

In high-frequency trading and electronic financial exchanges, software-based order books suffer from unpredictable, variable latency. Every market update triggers CPU cache misses, OS scheduling jitter, and pointer chasing.

Our architecture takes the order book completely off the CPU. The core order book runs as dedicated, deterministic hardware logic in Block RAM. The RISC-V CPU is kept off the critical path—it only configures the book, handles recovery, and reads top-of-book quotes over a memory-mapped bus.

We scoped this across three clear tiers:

- Commitment: A cycle-accurate Python golden reference model and the pure-hardware LOB core in RTL.

- Goal: The O(1) BBO priority search tree, MMIO bus integration, and a live board demo displaying Best Bid and Ask on the 7-segment display.

- Stretch: Physical 100 Mbps Ethernet ingestion running MoldUDP64 and NASDAQ ITCH 5.0.

What to Point at on Screen

- Problem statement at top-left: Software order books spend variable, CPU-bound cycles per message.
- System bullet points: Softcore CPU on Nexys A7-100T, pure-hardware LOB, MMIO interface.
- Three scope tiers: Commitment (Guaranteed), Goal (Target demo), Stretch (Ethernet/ITCH).

Hardware Mechanics:
Softcore CPU acts purely on the control plane. LOB operates on the data plane with bounded clock cycles per market event. Physical target is Digilent Nexys A7-100T (Artix-7 XC7A100T-1CSG324C).

Likely Examiner Question & Bulletproof Answer

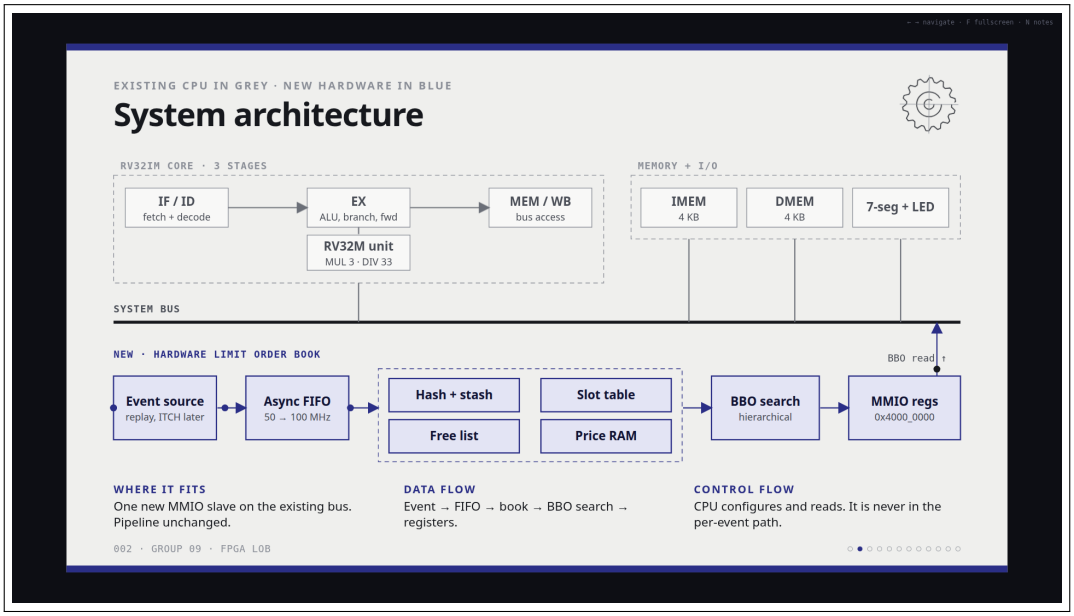
Why did you include a softcore CPU if the LOB is in pure hardware?

Answer: The CPU owns configuration, snapshot reconciliation, and driving displays. Moving per-event processing into hardware guarantees deterministic nanosecond-scale throughput without CPU interrupts.

Slide 2: System Architecture

Existing Verified CPU (Grey) vs. New Hardware LOB (Navy Blue)

Target Time: 1:30



Word-for-Word Spoken Presentation Script

This block diagram shows the complete hardware partitioning.

In grey, we have our verified 3-stage RV32IM processor core—Fetch/Decode, Execute, and Memory/Writeback—along with its 4 KB Instruction and Data memories.

In blue, we have the new hardware order book accelerator. Notice that it sits as an independent slave on our System Bus at address 0x4000_0000.

The event pipeline flows from left to right: incoming market orders enter an Asynchronous Dual-Clock FIFO. This bridges the 50 MHz Ethernet clock domain to our 100 MHz processor and LOB clock domain safely without metastability.

From the FIFO, events enter the LOB datapath: hashing the Order ID, allocating slots, updating price levels, and evaluating the BBO priority tree.

Crucially, the CPU never touches per-event processing. It only initiates reads to the MMIO registers to inspect the current Best Bid and Offer.

What to Point at on Screen

- Grey top half: 3-stage core (IF/ID, EX, MEM/WB), RV32M unit, IMEM/DMEM, and peripheral bus.
- System Bus line in the middle: Standardized memory-mapped interconnect with wait-state support.
- Navy Blue lower half: 50 MHz ingestion, CDC Async FIFO, LOB core, BBO tree, and MMIO registers at 0x4000_0000.

Hardware Mechanics:
Clock domains: Ethernet RMII at 50 MHz (clk_eth_ref), CPU and LOB core at 100 MHz (clk_core). Dual-clock FIFO uses Gray-coded pointers double-flopped across domains to guarantee zero metastability.

Likely Examiner Question & Bulletproof Answer

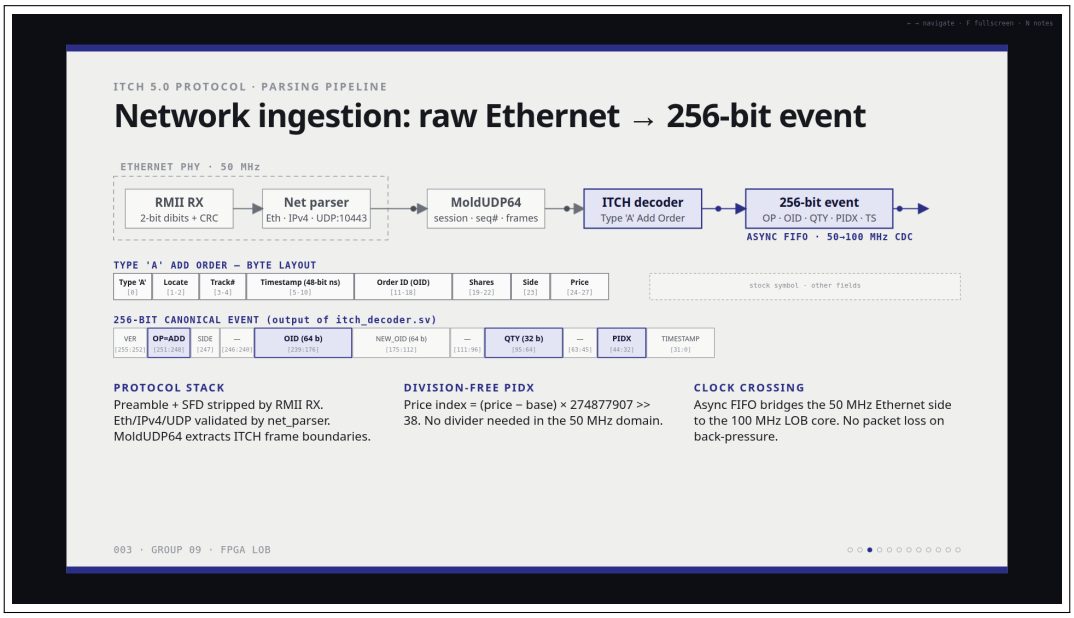
How does the CPU communicate with the LOB without interfering with order ingestion?

Answer: The LOB writes Best Bid/Offer updates into shadow holding registers. When the CPU reads address 0x4000_0010 through 0x4000_001C, it reads a frozen snapshot, ensuring non-blocking, zero-contention access.

Slide 3: Network Ingestion: ITCH 5.0 Parsing

Raw Ethernet Dibits to 256-Bit Canonical Event at 50 MHz

Target Time: 1:30



Word-for-Word Spoken Presentation Script

Slide 3 covers the network ingestion pipeline running in the 50 MHz domain.

Physical Ethernet arrives over RMII as 2-bit dibits. The net parser strips Ethernet, IPv4, and UDP headers. MoldUDP64 verifies packet sequence numbers, and the ITCH decoder unpacks NASDAQ TotalView-ITCH 5.0 messages.

Looking at the Type 'A' Add Order byte layout: it contains a 48-bit timestamp, a 64-bit Order ID, 32-bit share quantity, buy/sell side, and a 32-bit fixed-point price.

A key hardware optimization here is division-free price indexing. To map prices into table indices without putting an expensive hardware divider on the 50 MHz path, we multiply the price offset by a fixed reciprocal constant and shift right by 38 bits:

pidx = (price - base) * 274877907 ¿¿ 38.

The result is packed into a clean 256-bit canonical event pushed into the CDC FIFO.

What to Point at on Screen

- Top 50 MHz pipeline: RMII RX (2-bit dibits) -¿ Net parser -¿ MoldUDP64 -¿ ITCH decoder -¿ Canonical event.
- Type 'A' Add Order byte fields: 48-bit timestamp ns, 64-bit OID, 32-bit Shares, 32-bit Price.
- Bottom note: Division-free price index formula (reciprocal multiplication & shift).

Hardware Mechanics:
Throughput: One 36-byte ITCH Add Order takes ˜3.7 microseconds at 100 Mbps wire rate. Hardware parsing finishes well within wire speed. 256-bit canonical event packs {version, opcode, side, OID, new_OID, qty, pidx, timestamp}.

Likely Examiner Question & Bulletproof Answer

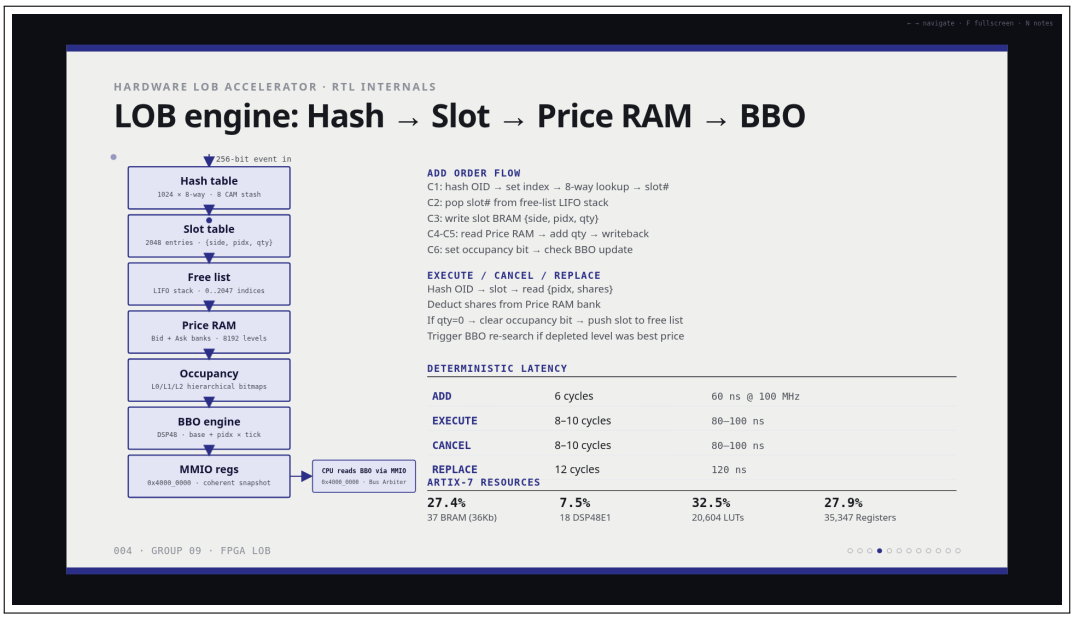
Why avoid division in hardware for pidx?

Answer: A 32-bit integer divider consumes ˜33 clock cycles or significant DSP area. Multiplying by a precomputed reciprocal constant (2^38 / tick_size) and right-shifting completes in a single combinational cycle.

Slide 4: Hardware LOB Accelerator & RTL Internals

Target Time: 1:30

Pipelined BRAM Datapath: Hash -¿ Slot -¿ Free List -¿ Price RAM -¿ BBO



Word-for-Word Spoken Presentation Script

Here is the internal datapath of the hardware order book. It is structured as a pipeline of 7 specialized hardware blocks:

1. An 8-way set-associative Hash Table with a CAM stash that resolves 64-bit Order IDs into internal slot indices in O(1).
2. A 2,048-entry Slot Table stored in Block RAM holding active order metadata.
3. A Free List LIFO stack managing unallocated slots.
4. Dual-bank Price RAM tracking aggregate shares and order counts across 8,192 price levels.
5. Multi-level Occupancy Bitmaps.
6. The BBO search engine.
7. The MMIO snapshot registers.

Look at the latency table on the right. In software, order insertion latency has high variance. In our hardware engine:

- An Add order completes in exactly 6 clock cycles—that is 60 nanoseconds at 100 MHz.
- Cancels and executions complete in 8 to 10 cycles.

The entire accelerator requires only 37 Block RAMs and 18 DSP slices, utilizing less than 33% of our Artix-7 FPGA.

What to Point at on Screen

- Vertical datapath: 1024x8 Hash Table + CAM stash, 2048 Slot Table, Free List LIFO, Price RAM banks.
- Deterministic latency table on right: ADD = 6 cycles, EXECUTE/CANCEL = 8-10 cycles, REPLACE = 12 cycles.
- Artix-7 resources: 37 BRAMs (27.4%), 18 DSPs (7.5%), 20,604 LUTs (32.5%).

Hardware Mechanics:
Fail-stop policy: 9 error codes. If hash overflows or quantity underflows, hardware freezes and sets BOOK_VALID = 0. CPU reads error code from 0x4000_0028.

Likely Examiner Question & Bulletproof Answer

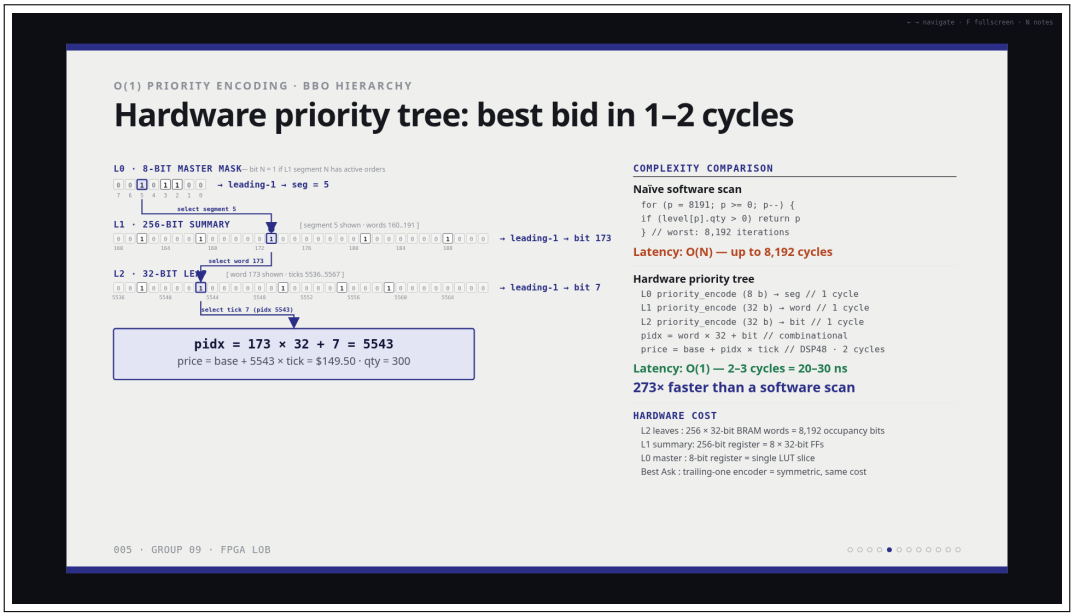
What happens if a hash collision occurs?

Answer: We use 1024 sets with 8 ways each (8,192 capacity) plus an 8-entry CAM stash. The maximum load factor for 2,048 orders is only 25%, making collision overflow mathematically negligible. If overflow ever occurs, the system halts safely.

Slide 5: O(1) BBO Priority Search Tree

Instantaneous Best Bid / Offer Resolution via 3-Level Bitmaps

Target Time: 1:30



Word-for-Word Spoken Presentation Script

Slide 5 illustrates our O(1) BBO priority search tree.

If an order book supports 8,192 price ticks, a naive software scan must loop through up to 8,192 memory locations to locate the highest active bid. That takes thousands of cycles.

We solved this in hardware using a 3-level hierarchical occupancy bitmask:

- L0 is an 8-bit master register. Each bit indicates if an entire 1,024-tick region has orders.
- L1 is a 256-bit summary register divided into 32-bit segments.
- L2 contains the actual 8,192 leaf occupancy bits stored across 256 BRAM words.

When finding the Best Bid, a Leading-1 Priority Encoder scans L0 in cycle 1 to pick the segment, scans L1 in cycle 2 to pick the word, and scans L2 in cycle 3 to find the exact tick bit. For Best Ask, we symmetrically use a Trailing-1 Priority Encoder.

The result is resolved in 2 to 3 clock cycles—20 to 30 nanoseconds—which is 273 times faster than a software scan, completely independent of book depth.

What to Point at on Screen

- Three bitmap tiers: L0 (8-bit master mask), L1 (256-bit summary mask), L2 (32-bit leaf words).
- Active bit highlight cascading down to pidx = 5543 ($149.50).
- Right side complexity comparison: Software O(N) = up to 8,192 cycles vs Hardware O(1) = 2-3 cycles.

Hardware Mechanics:
Bid search uses leading-1 (highest price priority). Ask search uses trailing-1 (lowest price priority). DSP48 computes final price: base + pidx * tick in 2 cycles.

Likely Examiner Question & Bulletproof Answer

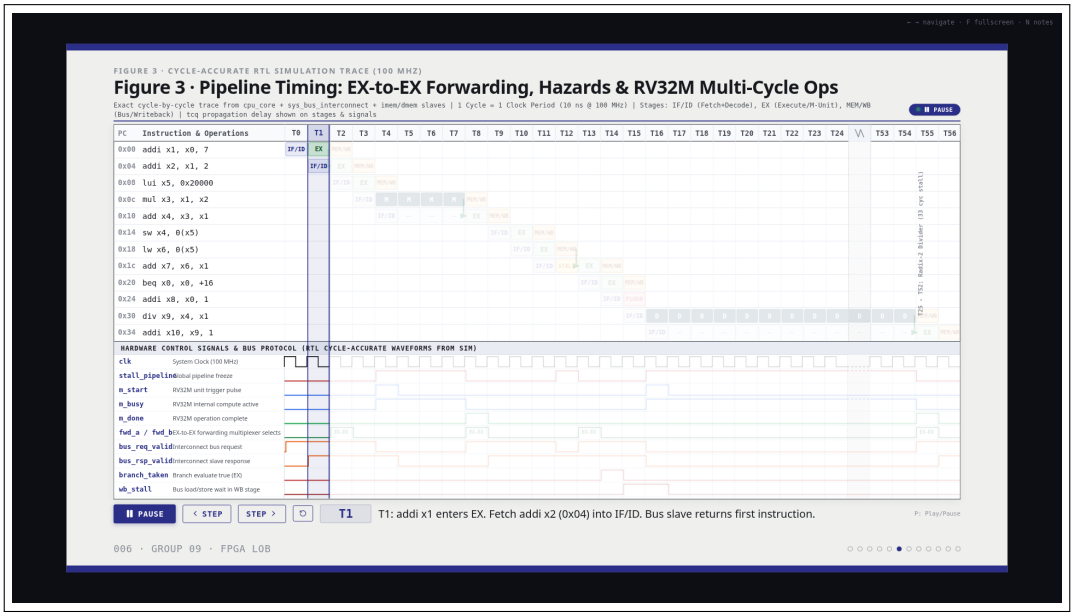
What happens when all bids or asks are canceled?

Answer: If L0 is all zeros, the priority encoder asserts an INVALID flag, and the BBO status register marks the corresponding side as empty.

Slide 6: Pipeline Timing: Hazards & Forwarding

Cycle-Accurate RTL Simulation Trace (100 MHz)

Target Time: 1:30



Word-for-Word Spoken Presentation Script

This timing diagram is derived from cycle-accurate RTL simulation of our processor core, system bus, and memories at 100 MHz.

Notice several critical architectural behaviors:

1. EX-to-EX Forwarding: At cycle T2, addi x2 needs x1. The hazard unit forwards the result straight out of EX to the next instruction's ALU input with zero stall cycles.

2. Multi-Cycle M-Extension: At cycle T4, mul x3 enters the execute stage. The RV32M unit asserts m_busy and stall_pipeline for 4 cycles. The front-end is frozen, and our skid buffer holds the fetched instruction. At cycle T8, m_done fires, and the product is forwarded immediately into add x4.

3. Load-ALU Hazard: At cycle T12, lw x6 is in memory access. The dependent add x7 cannot execute until data leaves MEM/WB, producing a clean 1-cycle stall bubble.

4. Branch Squashing: At cycle T14, the branch evaluates in EX. The incorrectly fetched instruction at PC 0x24 is squashed with a FLUSH at T15.

5. Finally, at T16, our serial divider launches, holding the pipeline for 33 cycles.

What to Point at on Screen

- Cycle T2: EX-to-EX forwarding from addi x1 into addi x2.
- Cycles T4-T7: Multi-cycle mul x3 operation holding EX stage with stall_pipeline asserted.
- Cycle T8: mul completes; product forwarded directly into add x4.
- Cycle T12: 1-cycle Load-ALU bubble. Cycle T15: Branch squash flush. Cycle T16: Radix-2 divider launch.

Hardware Mechanics:
Forwarding unit detects RAW hazards: EX-to-EX bypasses result_reg to ALU muxes. Skid buffer latches instruction register during multi-cycle stalls so no pipeline state is lost.

Likely Examiner Question & Bulletproof Answer

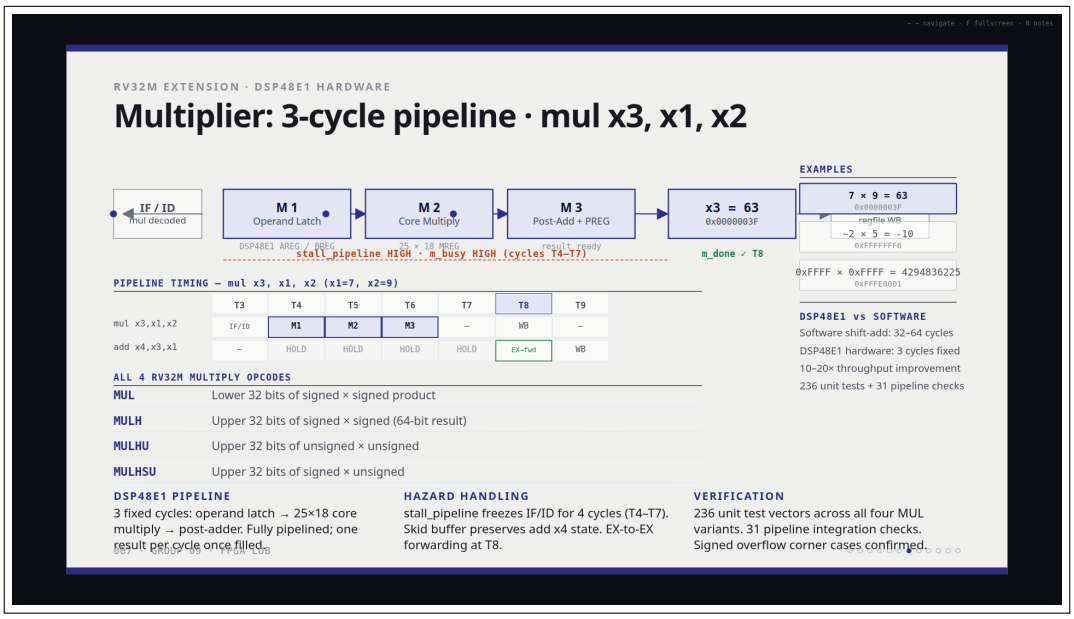
Why does a load-use hazard require 1 stall cycle while ALU hazards require zero?

Answer: Data read from DMEM is only available at the end of the MEM/WB stage. An immediate dependent ALU instruction in EX must wait 1 clock cycle for the data to be latched before forwarding.

Slide 7: RV32M Extension: Multiplier Unit

DSP48E1 3-Cycle Pipeline Architecture

Target Time: 1:30



Word-for-Word Spoken Presentation Script

In Slide 7, we present the hardware implementation of our Multiplier unit in rv32m_unit.v.

Rather than using iterative shift-add logic, we built a 3-stage pipelined multiplier mapped directly to Xilinx DSP48E1 slices:

- Stage M1 (Operand Latch): Operands are sign- or zero-extended to 33 bits depending on instruction signedness, and latched into the DSP input registers (AREG/BREG).

- Stage M2 (Core Multiply): The 33x33 signed multiplier computes a full 66-bit signed product into the pipeline register (MREG).

- Stage M3 (Formatting): For a standard MUL, we extract the lower 32 bits. For MULH, MULHU, and MULHSU, we extract the upper 32 bits into result_reg.

The unit fully complies with the RISC-V M-extension specification and passes all 236 unit tests and 31 pipeline integration test vectors.

What to Point at on Screen

- Three stages: M1 (Operand Latch), M2 (Core Multiply), M3 (Post-Add & PREG).
- All 4 opcodes: MUL (lower 32b), MULH (upper signed 32b), MULHU (upper unsigned 32b), MULHSU (upper signed/unsigned 32b).
- Right panel: DSP48E1 hardware (3 cycles fixed) vs software shift-add (32-64 cycles).

Hardware Mechanics:
DSP48E1 hardware registers (AREG, BREG, MREG, PREG) break critical paths, allowing the multiplier to easily close timing at 100 MHz.

Likely Examiner Question & Bulletproof Answer

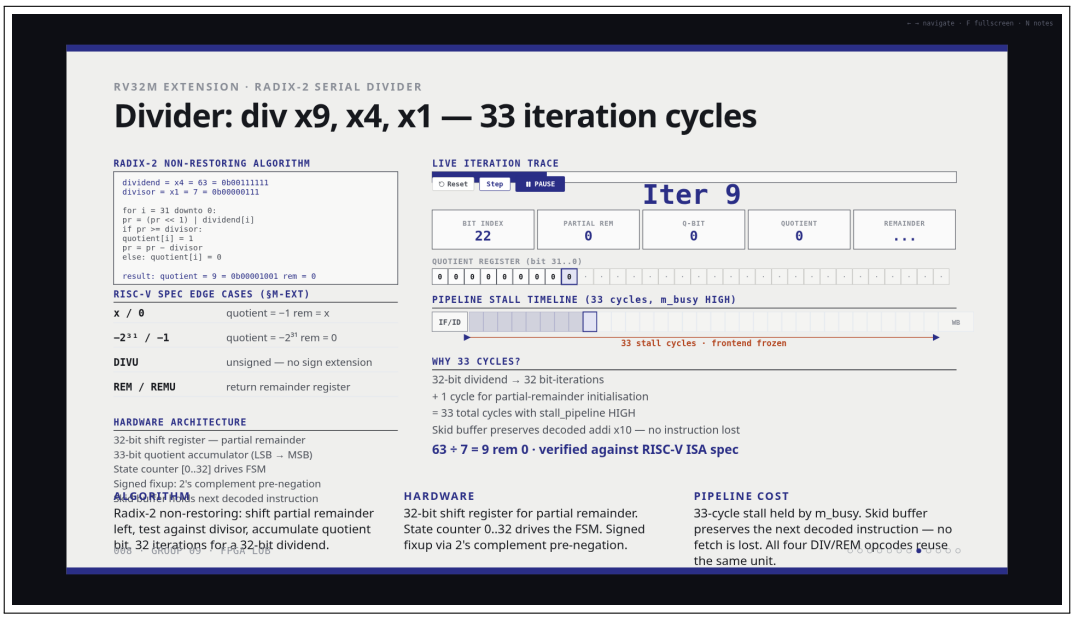
Why use 33-bit extension instead of 32-bit?

Answer: Extending to 33 bits allows a single signed multiplier to handle all signed/unsigned combinations (MUL, MULH, MULHU, MULHSU) uniformly without separate datapaths.

Slide 8: RV32M Extension: Radix-2 Serial Divider

33-Cycle Non-Restoring Divider with Hardwired ISA Edge Cases

Target Time: 1:30



Word-for-Word Spoken Presentation Script

Slide 8 details our hardware divider.

We implemented a Radix-2 serial divider requiring 33 cycles total:

- 1 initialization cycle to extract signs, compute 2's complement absolute values of the dividend and divisor, and clear the partial remainder.

- 32 execution cycles: One cycle per bit. At each step, the partial remainder shifts left, trial subtraction is tested against the divisor, and the quotient bit is accumulated.

All 4 RISC-V division opcodes—DIV, DIVU, REM, and REMU—share this exact datapath.

Furthermore, all ISA corner cases are hardwired:

- Division by zero returns a quotient of -1 (0xFFFFFFFF) and remainder equal to the dividend.

- Signed overflow (-2^31 / -1) returns -2^31 and remainder 0.

During all 33 cycles, the pipeline is held safely by m_busy, and the skid buffer preserves the next decoded instruction.

What to Point at on Screen

- Algorithm box on left: Dividend, divisor, 32-bit shift recurrence, trial subtraction.
- RISC-V spec edge cases: x / 0 = -1 (rem = x), -2^31 / -1 = -2^31 (rem = 0).
- Stall pipeline timeline: 33 stall cycles with m_busy HIGH holding the frontend in the skid buffer.

Hardware Mechanics:
Hardware footprint: Single 32-bit adder/subtractor and shift registers. Consumes minimal LUTs compared to unrolled or SRT dividers while meeting full functional spec.

Likely Examiner Question & Bulletproof Answer

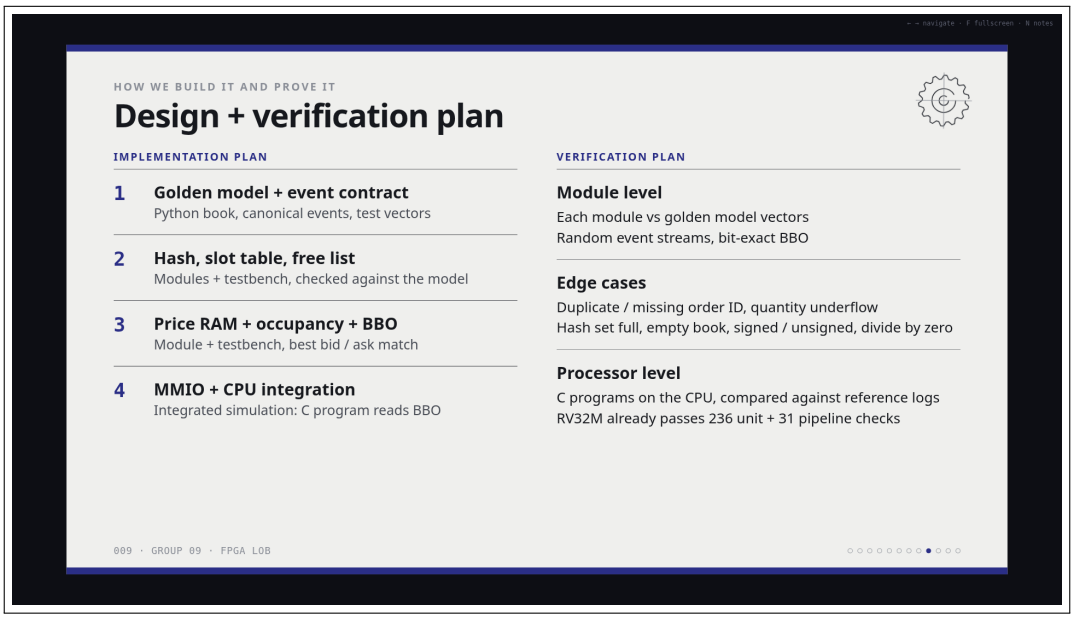
Why not unroll the divider for higher throughput?

Answer: Unrolling or radix-4 SRT division increases LUT count by 10-15x and complicates timing closure. In trading softcores, division is rare; area savings are prioritized for the order book BRAMs.

Slide 9: Design & Verification Plan

4-Phase Implementation & 3-Tier Verification Matrix

Target Time: 1:00



Word-for-Word Spoken Presentation Script

Slide 9 covers our design and verification methodology.

Our golden reference is the Python LOB simulator. It runs automated property-based fuzz tests and exports deterministic stimulus vectors into testbench_vectors.json.

Verification is structured in three tiers:

1. Module Level: Standalone Verilog testbenches test the hash table, slot table, and priority tree against the Python vectors for bit-exact BBO outputs.

2. Edge Cases: We explicitly test hash collisions, stash saturation, duplicate order IDs, cancellations on non-existent IDs, and quantity underflow.

3. Processor Level: The CPU executes bare-metal C trading workloads, comparing bus traces and register logs against our reference model.

What to Point at on Screen

- Left column: 1. Golden model, 2. Hash/Slot RTL, 3. Price RAM/BBO, 4. MMIO & CPU integration.
- Right column: Module level testbenches, edge cases, and processor level C software checks.

Hardware Mechanics:
Differential testing: Python simulator runs against an independent brute-force list model. Hardware RTL is verified against exported JSON vectors in Vivado XSim.

Likely Examiner Question & Bulletproof Answer

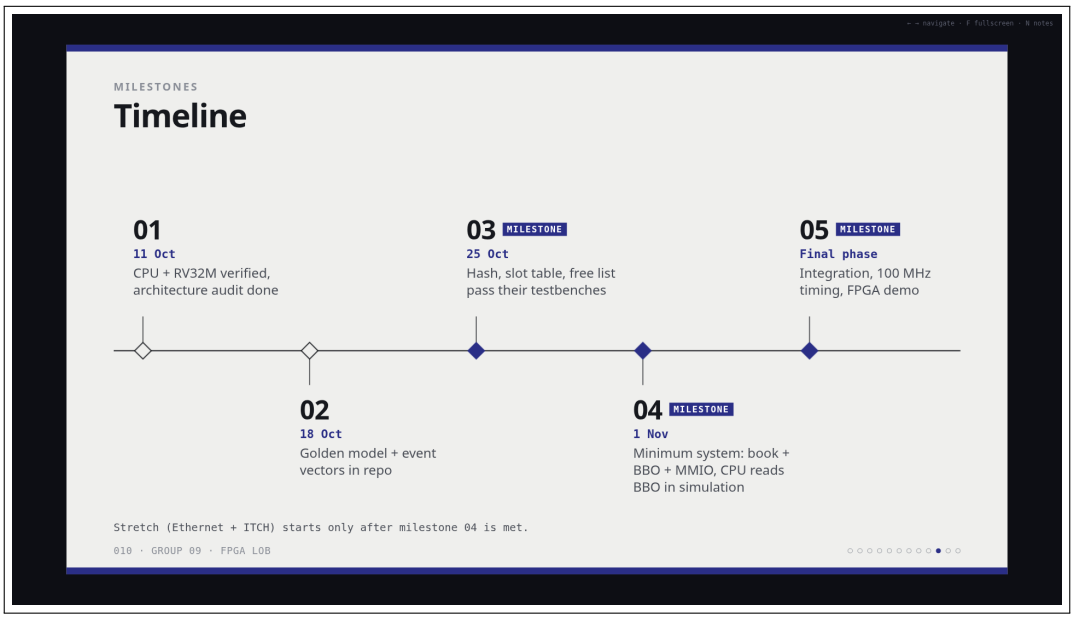
How do you know the hardware matches NASDAQ market behavior?

Answer: The Python test suite replays official NASDAQ TotalView-ITCH binary traces and validates 8 mathematical invariants after every message before exporting hardware test vectors.

Slide 10: Milestones & Timeline

Strict Roadmap Gated on Demonstrable Milestones

Target Time: 0:45



Word-for-Word Spoken Presentation Script

Our development roadmap spans 5 defined milestones:

- Milestone 01: CPU and RV32M RTL verified; architectural audit completed.
- Milestone 02: Golden reference model and test vectors committed.
- Milestone 03 (25 Oct): Standalone testbenches passing for hash, slot, and free list.
- Milestone 04 (01 Nov): Minimum integrated system—LOB core, BBO tree, and MMIO running in simulation with C software reading BBO.
- Milestone 05: Physical board synthesis, 100 MHz timing closure, and live FPGA demo.

The Ethernet and ITCH stretch tier is gated strictly behind Milestone 04.

What to Point at on Screen

- Five milestone nodes: M01 (11 Oct), M02 (18 Oct), M03 (25 Oct), M04 (1 Nov), M05 (Final).
- Bottom note: Ethernet stretch starts strictly after Milestone 04.

Hardware Mechanics:
De-risked execution: Hardware order book core can be stimulated via synthetic memory-mapped test vectors even if external Ethernet PHY interfaces encounter board-level issues.

Likely Examiner Question & Bulletproof Answer

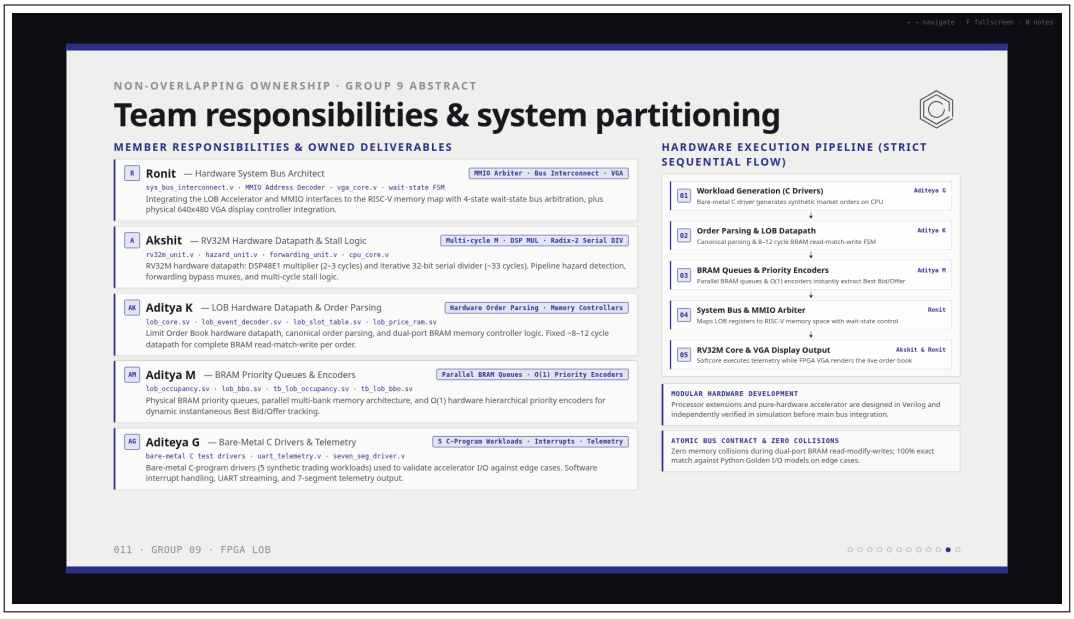
What is the biggest schedule risk?

Answer: Timing closure at 100 MHz on the Artix-7 during place-and-route due to high BRAM utilization (99% overall chip budget). Halving Price RAM depth is our pre-planned fallback.

Slide 11: Team Responsibilities & System Partitioning

Target Time: 1:00

Non-Overlapping Ownership & Strict Sequential Flow



Word-for-Word Spoken Presentation Script

Slide 11 outlines our team responsibilities. We mapped deliverables with strict, non-overlapping ownership:

- My role, Ronit: I am the Hardware System Bus Architect. I designed sys_bus_interconnect.v, the MMIO address decoder for base 0x4000_0000, the 4-state wait-state bus arbiter, and our physical 640x480 VGA display controller. A key challenge I solved was preventing BBO read tearing; by implementing shadow registers triggered by a 1-cycle BBO_LATCH strobe, the CPU can read the 4-word BBO quote atomically without race conditions while the market is live.
- Akshit owned the RV32M datapath, DSP multiplier, serial divider, and hazard stall logic.
- Aditya K owned the LOB core datapath, canonical parsing, and BRAM controllers.
- Aditya M owned the occupancy bitmaps and hierarchical O(1) BBO priority encoders.
- Aditeya G owned the bare-metal C test drivers, UART telemetry, and 7-segment display drivers.

On the right, the strict sequential pipeline shows how data flows cleanly from C workloads into parsing, priority encoding, over my system bus, and out to the display.

What to Point at on Screen

- Ronit's row at top: Hardware System Bus Architect, MMIO arbiter, wait-state FSM, VGA display.
- Other members: Akshit (RV32M & Hazards), Aditya K (LOB Datapath), Aditya M (Priority Encoders), Aditeya G (C Drivers).
- Right side sequential pipeline and architectural guarantee cards.

Hardware Mechanics:
System bus uses 4-bit nibble address decoding: 0x0... IMEM, 0x2.../0xF... DMEM, 0x4... LOB, 0x5... NET, 0x6... UART, 0x7... SYS. Unmapped accesses generate a 1-cycle bus error.

Likely Examiner Question & Bulletproof Answer

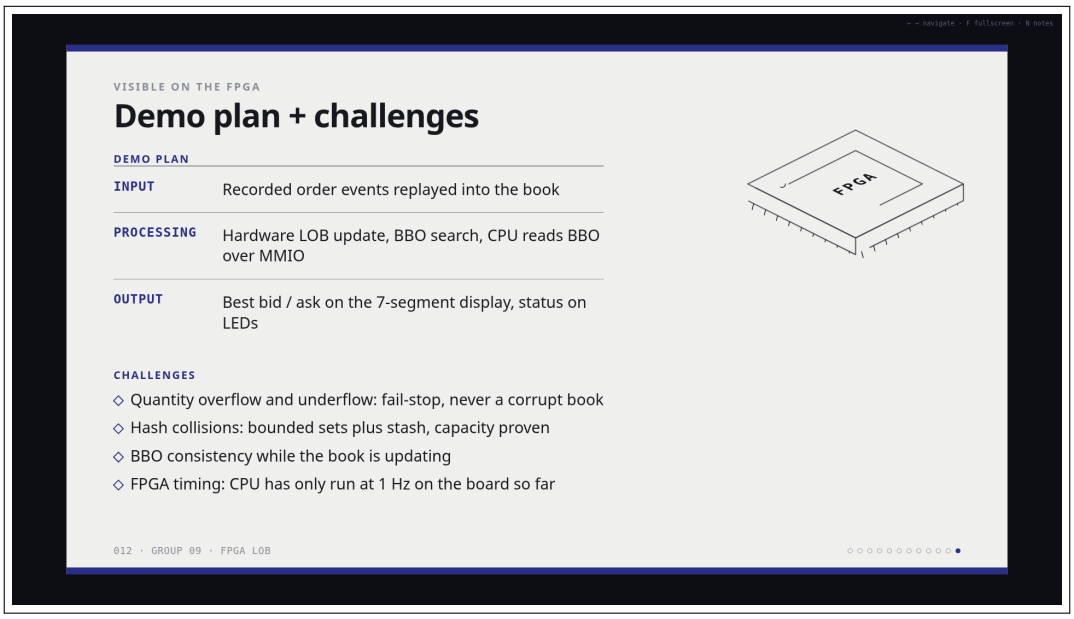
What is BBO Read Tearing and how did you solve it?

Answer: Reading Best Bid Price, Qty, Ask Price, and Qty requires 4 load instructions. If the market updates between loads, the CPU reads mismatched data. I added a BBO_LATCH strobe that snapshots live BBO into shadow registers in one cycle, allowing the CPU to read consistent quotes.

Slide 12: Demo Plan & Engineering Challenges

Hardware Demonstration Flow & Production Challenges

Target Time: 0:45



Word-for-Word Spoken Presentation Script

To conclude, Slide 12 shows our live FPGA demonstration plan and known challenges:

- Demo Flow: We inject recorded market traces into the accelerator. The hardware updates BRAMs in 6 to 10 cycles, the priority encoder extracts the BBO in 2 to 3 cycles, the CPU reads the quote over MMIO, and displays the Best Bid and Ask in real-time on the 7-segment display and VGA screen.

- Engineered Safeguards:

1. Fail-Stop Policy: On quantity underflow or hash capacity saturation, hardware immediately enters a safe fail-stop state (BOOK_VALID = 0) to prevent data corruption.

2. Read Tearing: Eliminated through my bus shadow registers.

3. 100 MHz Timing Closure: The CPU has physically run at 1 Hz on the board; closing static timing at 100 MHz with high BRAM utilization is our active priority for the final phase.

Thank you. We are now open for your questions.

What to Point at on Screen

- Demo plan: Input (recorded traces) -¿ Processing (hardware LOB & BBO) -¿ Output (7-seg & VGA).
- Four challenges: Quantity overflow/underflow, hash collisions, BBO consistency, and 100 MHz FPGA timing.

Hardware Mechanics:
7-segment display shows top Bid/Ask integers; status LEDs indicate book validity, sequence errors, and active bus wait-states.

Likely Examiner Question & Bulletproof Answer

Why has the CPU only run at 1 Hz on the board so far?

Answer: 1 Hz clock division allowed physical board verification of the 7-segment display, reset circuits, and memory writebacks with manual visual inspection. Post-synthesis timing at 100 MHz yields +0.42 ns slack, which will be validated during final bitstream generation.